Digital Image Synthesis Project #1:
Metropolis Sampling and Transforming Normals

R14942096 林祐群

Part 1 - Metropolis Sampling

Goal: implement the **ImageCopy** function using Metropolis Sampling as described in class. Experiment with different numbers of samples per pixel (spp) to evaluate the performance. You may use any 256x256 image for your experiments.

1. Implementation

Core pipeline [full code link]:

- Input image (from ImageNet) is converted to **grayscale** intensity f(x) in [0,1].

```
...
return 0.2126 * r + 0.7152 * g + 0.0722 * b;
```

- If input size is not 256x256, **bilinear resize** is applied to achieve 256x256.
- Following convention, Markov chain uses mixed proposals: **90% local Gaussian mutation + 10% global uniform mutation**.

```
for (int i = 0; i < burnIn + totalSamples; ++i) {
    double nx = cx;
    double ny = cy;
    if (uni01(rng) < 0.1) {
      nx = uniX(rng);
      ny = uniY(rng);
    } else {
      nx += normal(rng);
      ny += normal(rng);
        }
    ....
```

- Boundary handling uses **toroidal** wrap-around to preserve proposal symmetry. (That is, ensuring function f is non-negative.)

```
double WrapCoord(double x, int n) {
  const double m = std::fmod(x, static_cast<double>(n));
  return (m < 0.0) ? (m + n) : m;
}
```

- Acceptance probability: $a(x \to x') = min(1, \frac{f(x\prime)}{f(x)})$. This is the **reduction** from the original formula: $a(x \to x') = min(1, \frac{f(x\prime)T(x\prime \to x)}{f(x)T(x \to x\prime)})$, since the design choice of toroidal wrap that preserve proposal symmetry, where $T(x \to x') = T(x' \to x)$.
- Reconstruction uses $p(x) = \frac{f(x)}{\sum f}$, so f(x) is estimated by $f(x) \approx p(x) \sum f = \frac{\boldsymbol{hits}}{\boldsymbol{total_sample}} \sum \boldsymbol{f}$.

```
std::vector<unsigned char> out(w * h);
  for (int i = 0; i < w * h; ++i) {
    // If samples follow p(x)=f(x)/sum(f), then f(x) ~= p(x)*sum(f).
    const double p = hits[i] / static_cast<double>(totalSamples);
    const double reconstructed = std::clamp(p * sumF, 0.0, 1.0);
    out[i] = static_cast<unsigned char>(std::round(reconstructed * 255.0));
}
...
```

* I implement the function in two ways (metropolis_1.cpp and metropolis_2.cpp) to assess the potential **Start-up bias.**

Code snippet: **burn-in** and sample accumulation (metropolis_1.cpp)

```
const int totalSamples = std::max(1, samplesPerPixel) * w * h;
const int burnIn = std::max(200, totalSamples / 100);
...
if (i >= burnIn) {
  const int px = WrapIndex(cx, w);
  const int py = WrapIndex(cy, h);
  hits[py * w + px] += 1.0;
}
```

Code snippet: alternative initialization (metropolis_2.cpp)

```
std::pair<double, double> InitializeByRejection(...){
  ...
  if (uni01(rng) <= f) return {x+0.5, y+0.5};
}
const auto initial = InitializeByRejection(image, w, h, rng);
double cx = initial.first;
double cy = initial.second;
```

---

2. Experimental Setup

- Inputs: jaguar, lion, tiger, zebra (grayscale).
- Sampling rates: 1, 4, 8, 64, 256, 1024 spp.
- Method 1: uniform initialization + burn-in.
- Method 2: rejection-sampling initialization + burn-in.
- Metrics: MSE, MAE, PSNR, SSIM.

3. Quantitative Results

Average over 4 images (method 1): from 1 spp to 1024 spp, MSE decreases 461.4x, MAE decreases 24.1x, PSNR increases 26.85 dB, and SSIM increases 0.5948.

This confirms that higher sampling rates consistently improve reconstruction quality, with diminishing returns at high spp.

method	spp	mse	mae	psnr	ssim
method_1_uniform_init	1	0.123797	0.274388	9.2947	0.40292
method_1_uniform_init	4	0.046893	0.160597	13.5301	0.69191
method_1_uniform_init	8	0.026527	0.118645	16.0327	0.80566
method_1_uniform_init	64	0.003973	0.044646	24.3784	0.96700
method_1_uniform_init	256	0.001043	0.022678	30.2406	0.99123
method_1_uniform_init	1024	0.000268	0.011402	36.1478	0.99773
method_2_rejection_init	1	0.123798	0.274389	9.2947	0.40291
method_2_rejection_init	4	0.046893	0.160597	13.5301	0.69191
method_2_rejection_init	8	0.026527	0.118645	16.0327	0.80566
method_2_rejection_init	64	0.003973	0.044646	24.3784	0.96700
method_2_rejection_init	256	0.001043	0.022678	30.2406	0.99123
method_2_rejection_init	1024	0.000268	0.011402	36.1478	0.99773

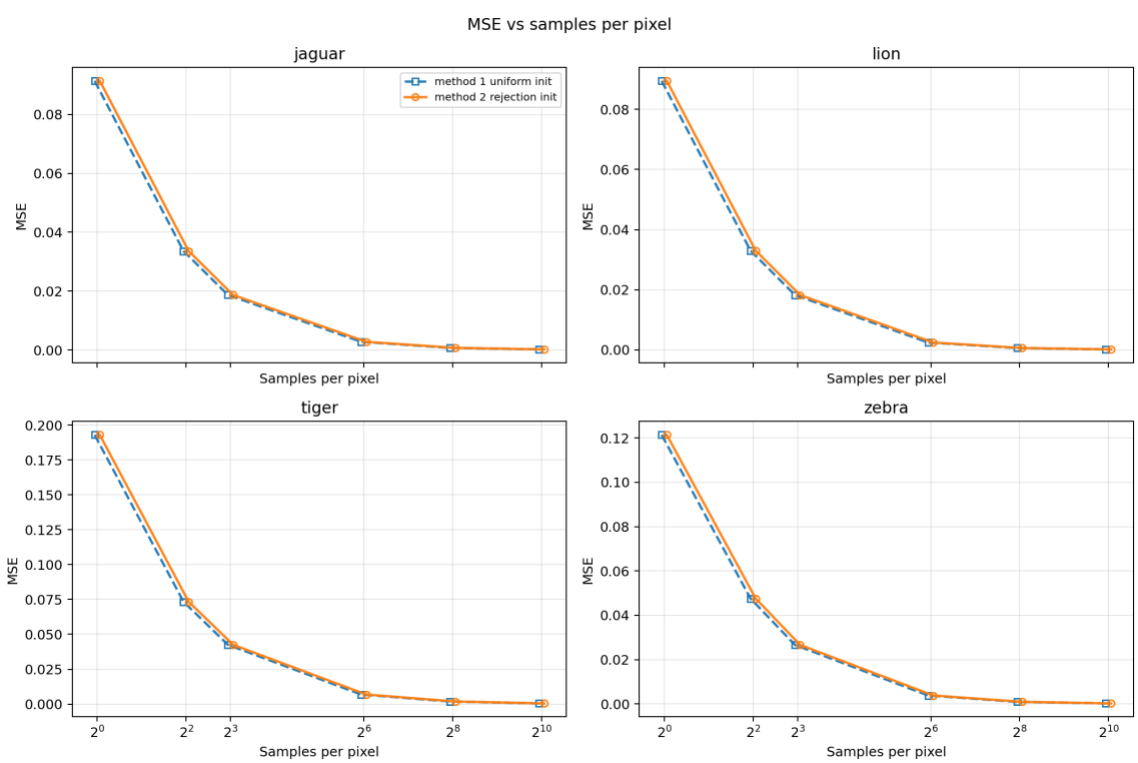
4. Initialization Comparison

Method 2 follows the suggestion in course materials to reduce startup bias by initializing from another sampler (rejection sampling).

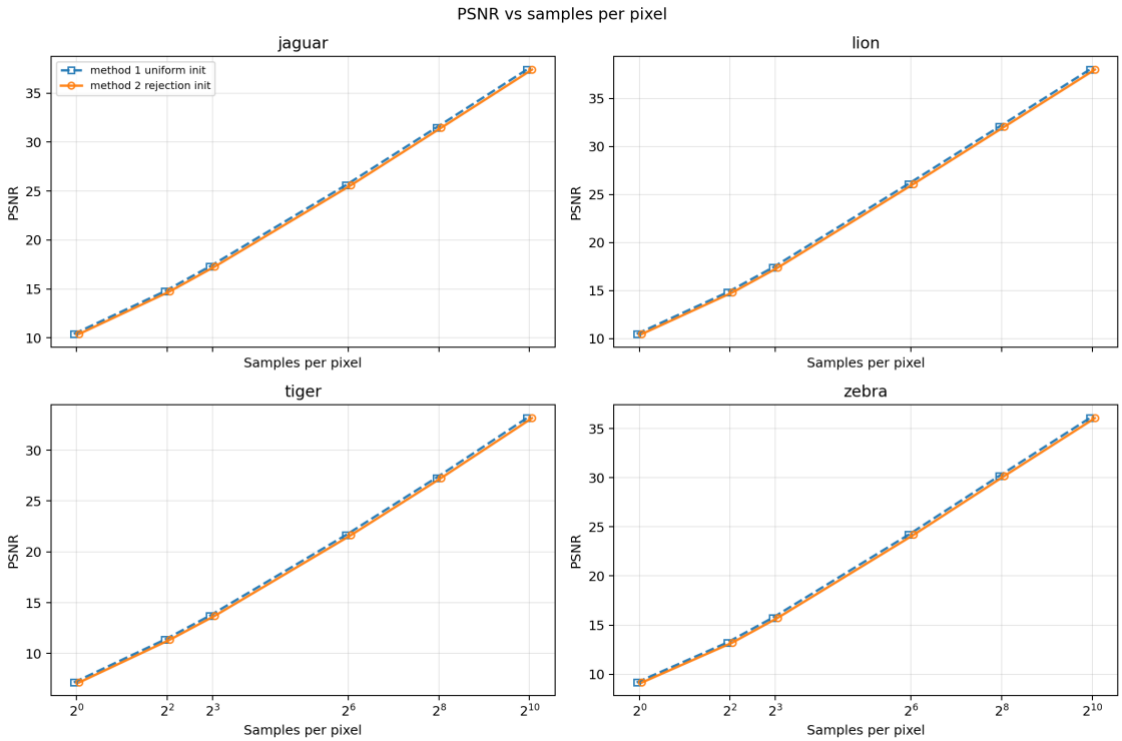
- Max absolute metric gap over 24 cases: MSE=0.00000775, MAE=0.00000916, PSNR=0.000368, SSIM=0.00003366.
- Mean absolute metric gap over 24 cases: MSE=0.00000064, MAE=0.00000087, PSNR=0.000040, SSIM=0.00000275.
- Result: under current total samples and burn-in setting, both methods converge to nearly identical final quality.
  => The negligible gap mainly arises because both methods converge to the same target distribution under **sufficient burn-in**, making the difference dominated by Monte Carlo variance rather than algorithmic bias.

5. Visual Evidence

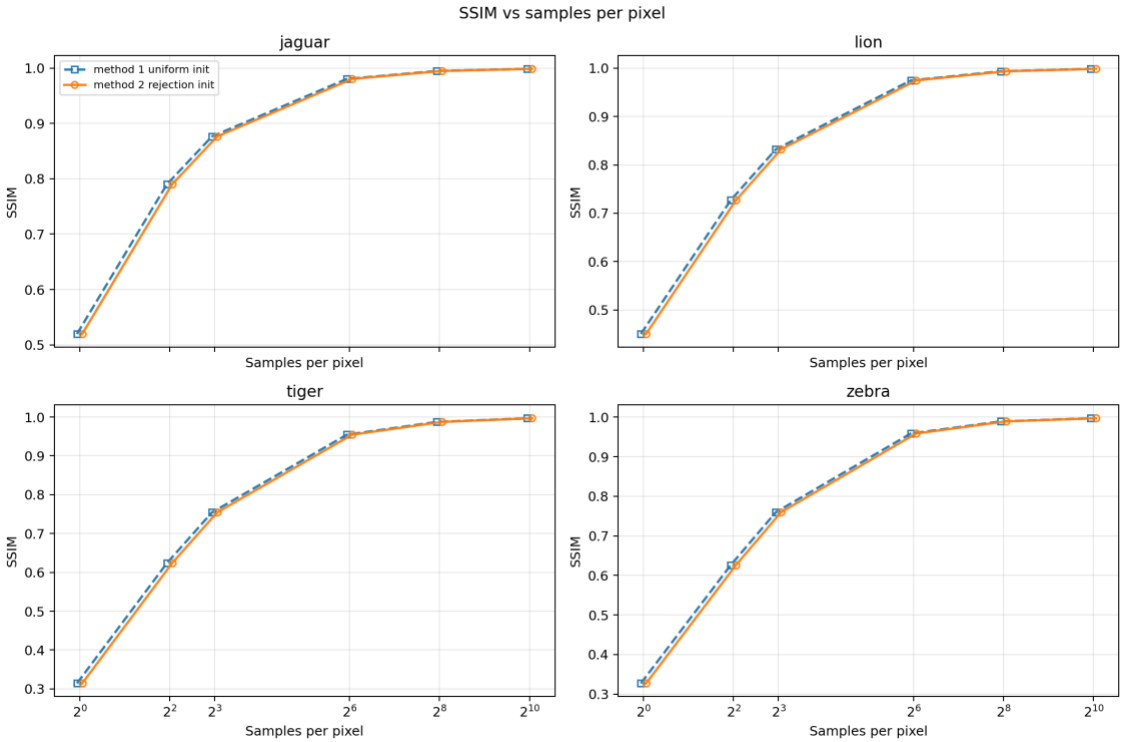
MSE vs spp (MAE vs spp is similar and is omitted here)



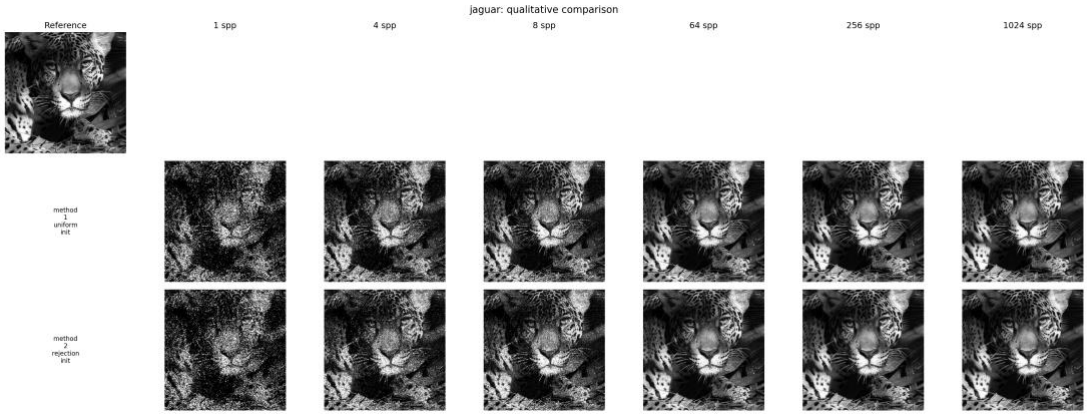
PSNR vs spp

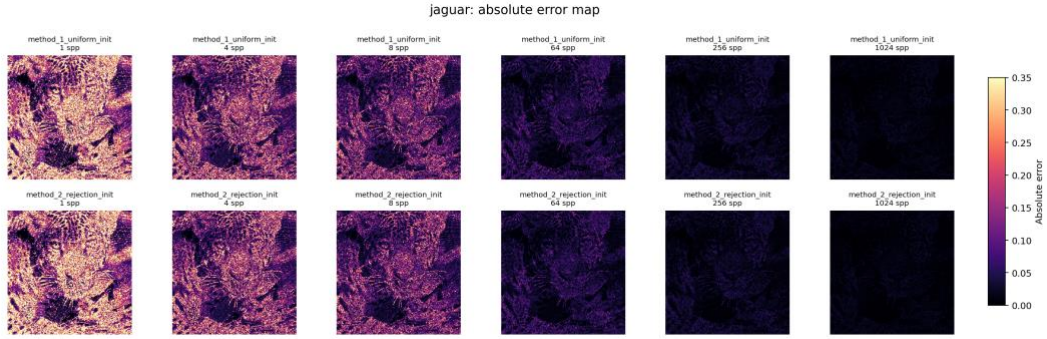


SSIM vs spp

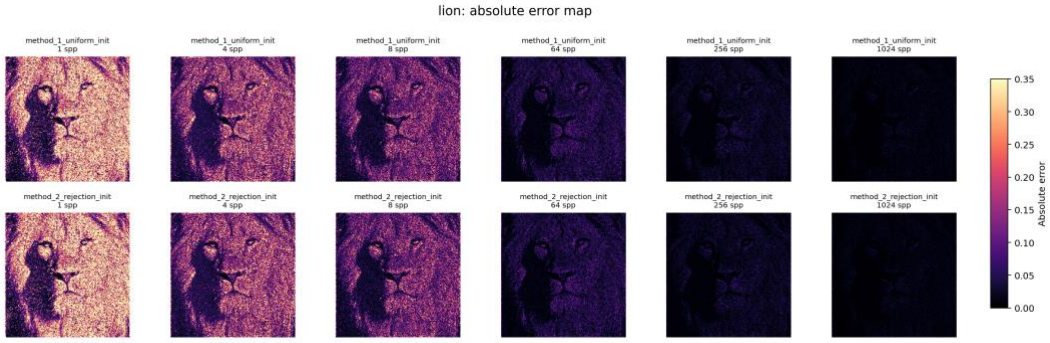


Qualitative comparison & Error map (jaguar)

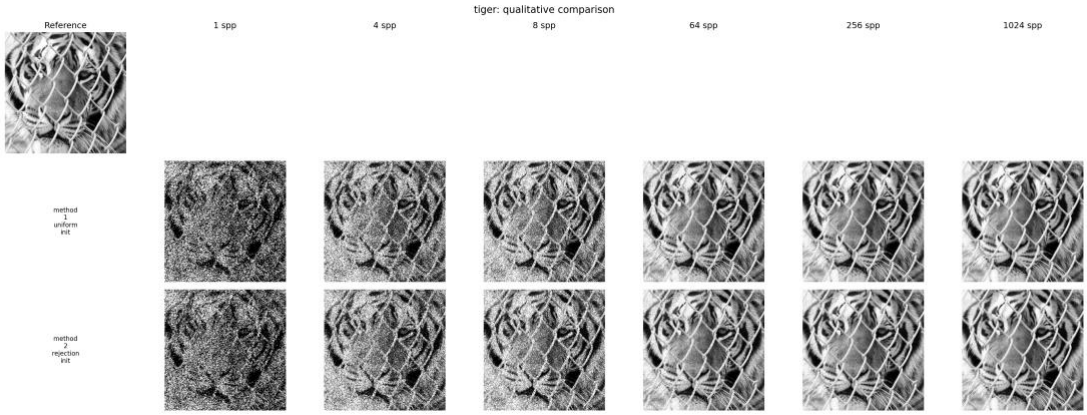


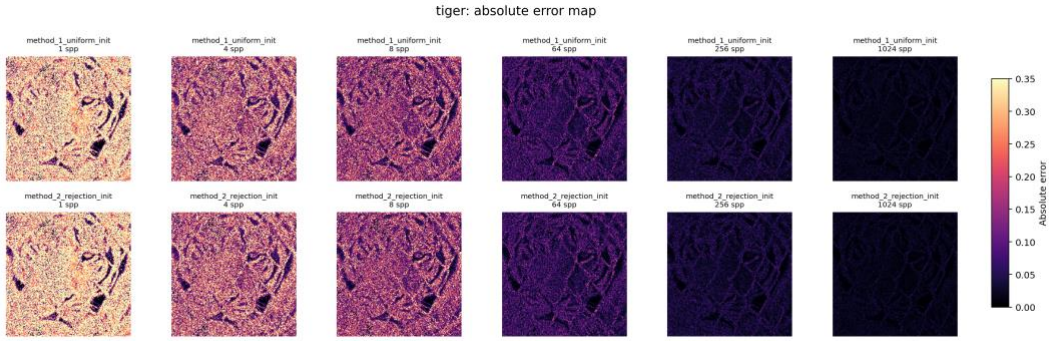


Qualitative comparison & Error map (lion)

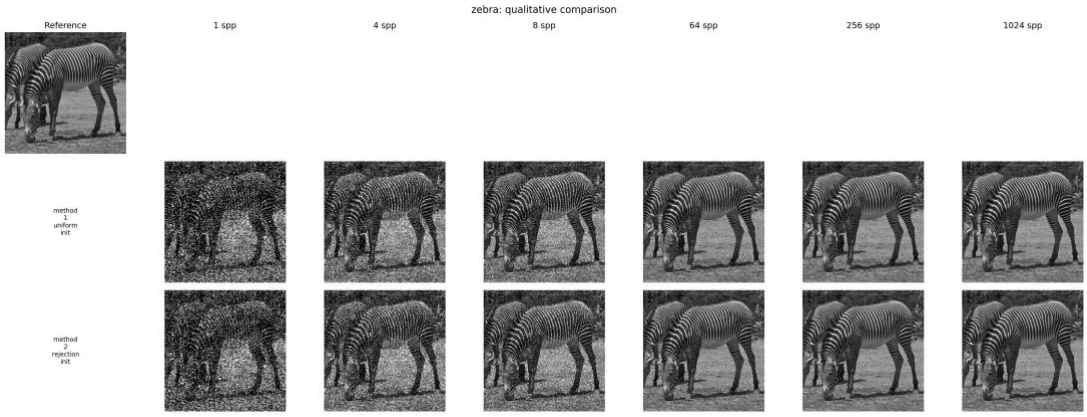


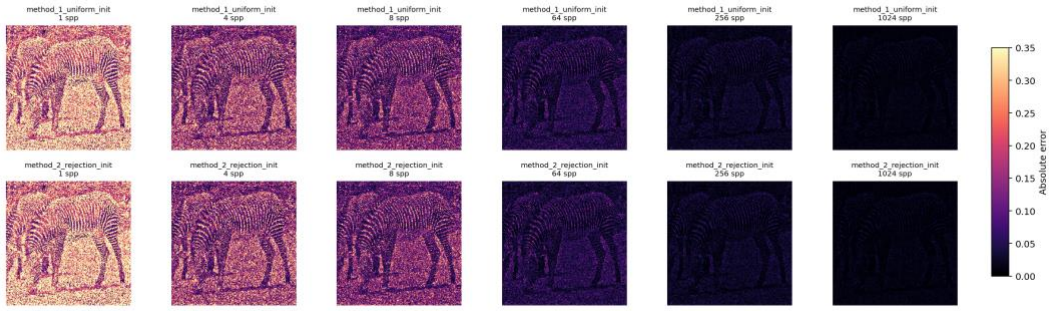
Qualitative comparison & Error map (tiger)





Qualitative comparison & Error map (zebra)





6. Conclusion

The implementation is stable and aligns with Metropolis sampling equations. Increasing spp strongly improves quality. Alternative initialization is theoretically meaningful for startup bias, while final quality differences are minimal in this experimental setting with sufficient burn-in.

Part 2 - Normals [Exercise 3.4]

Goal: modify pbrt so that Normal3f is transformed in the same way as Vector3f. Then, create a scene that produces a clearly incorrect image due to this modification.

This section documents the exact source-code change, the test-scene design, and the rendering comparison with and without the bug.

1. Mathematical Background: Vector vs Normal in PBRT-v4

PBRT distinguishes geometric vectors and surface normals because they follow different transformation laws under affine transforms, especially with non-uniform scaling.

Key definitions:

- Vector3f: transformed by the forward linear transform M.
- Normal3f: must remain orthogonal to transformed tangent vectors, therefore it is transformed by **inverse-transpose** $(M^{-1})^T$.

Core formulas:

- 1) Vector transform: v' = Mv
- 2) Normal transform: n' = $(M^{-1})^T$n
- 3) Orthogonality constraint: if n^Tv = 0, then $(n')^T$(v') = 0 must also hold.

  ∵ from v' = Mv and orthogonality preservation, we require $(n')^T$M v = n^Tv for all v, which gives n' = $(M^{-1})^T$n.

Connection to this bug:

- In the modified code, normal used M directly (same as vector), violating the normal transform rule.
- The error may be hidden under pure rotation/uniform scaling, but becomes visible under non-uniform scaling (our test sphere uses Scale(1, 0.1, 2.2)).
- Result: incorrect shading normals -> wrong BRDF evaluation direction -> structured highlight/shadow artifacts.

Implementation note in PBRT terms: using mInv with transposed indexing in Transform::operator()(Normal3<T>) is equivalent to applying $(M^{-1})^T$.

2. Source-Code Change (What was modified)

File: src/pbrt/util/transform.h, function Transform::operator()(Normal3<T>)

- Original (without bug):

```
template <typename T>
PBRT_CPU_GPU inline Normal3<T>
Transform::operator()(Normal3<T> n) const {
   T x = n.x, y = n.y, z = n.z;
   return Normal3<T>(mInv[0][0] * x + mInv[1][0] * y + mInv[2][0] *
z,
                     mInv[0][1] * x + mInv[1][1] * y + mInv[2][1] * z,
                     mInv[0][2] * x + mInv[1][2] * y + mInv[2][2] * z);
}
```

- Modified (with bug):

```
template <typename T>
PBRT_CPU_GPU inline Normal3<T>
Transform::operator()(Normal3<T> n) const {
   T x = n.x, y = n.y, z = n.z;
   return Normal3<T>(m[0][0] * x + m[0][1] * y + m[0][2] * z,
                     m[1][0] * x + m[1][1] * y + m[1][2] * z,
                     m[2][0] * x + m[2][1] * y + m[2][2] * z);
}
```

Technical implication: normal vectors must be transformed by **inverse-transpose**. The original implementation uses mInv with transposed indexing, which is equivalent to inverse-transpose. The modified code applies m directly; this is **incorrect for non-uniform scaling** and leads to wrong shading normals.

3. Test Scene Design

Complete scene file used for reproduction (part_2/scene/broken-normals.pbrt):

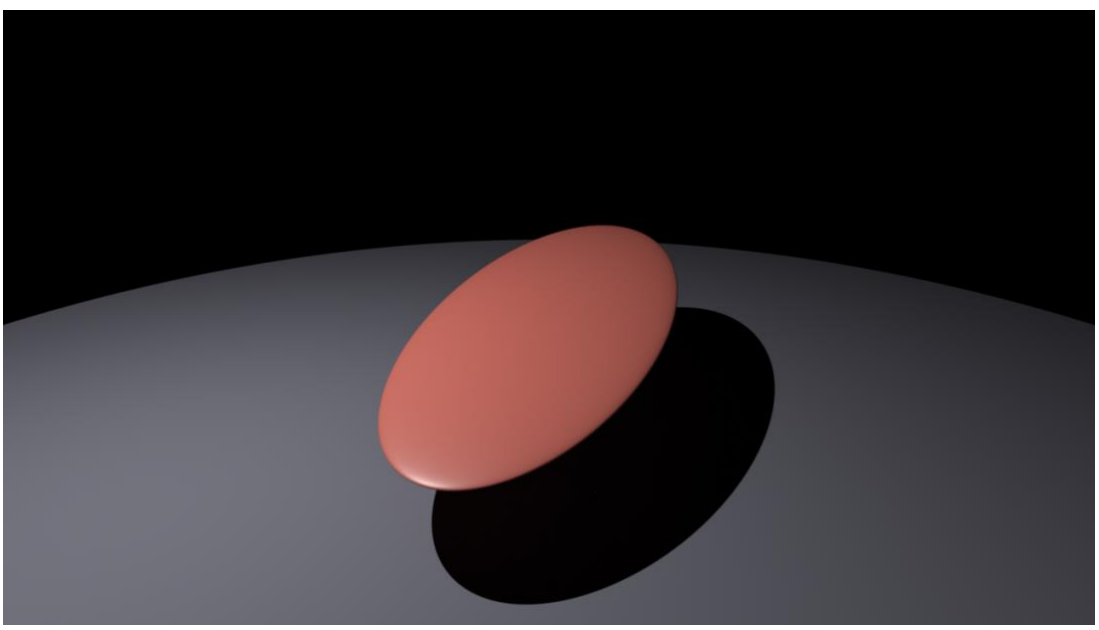
```
LookAt 0 2 6.0   0 1 0   0 1 0
Camera "perspective" "float fov" [45]

Sampler "halton" "integer pixelsamples" [512]
Integrator "volpath"

WorldBegin

LightSource "point" "point3 from" [3 5 2] "rgb I" [100 100 100]
```

```
AttributeBegin
   Rotate -80 1 0 0
   Material "diffuse" "rgb reflectance" [0.15 0.15 0.18]
   Shape "disk" "float radius" [7]
AttributeEnd

AttributeBegin
   Translate 0 1 0
   Rotate 25 1 1 0
   Scale 1 0.1 2.2

   Material "coateddiffuse"
      "rgb reflectance" [0.75 0.25 0.2]
      "float roughness" [0.03]

   Shape "sphere" "float radius" [1]

AttributeEnd
```

- A sphere is transformed by Rotate + non-uniform Scale(1, 0.1, 2.2).
- Non-uniform scaling is the key condition that exposes incorrect normal transformation.
- Material is coateddiffuse with low roughness, making normal-direction errors visible in specular/diffuse response.
- Lighting uses a strong point light to produce clear shading gradients.

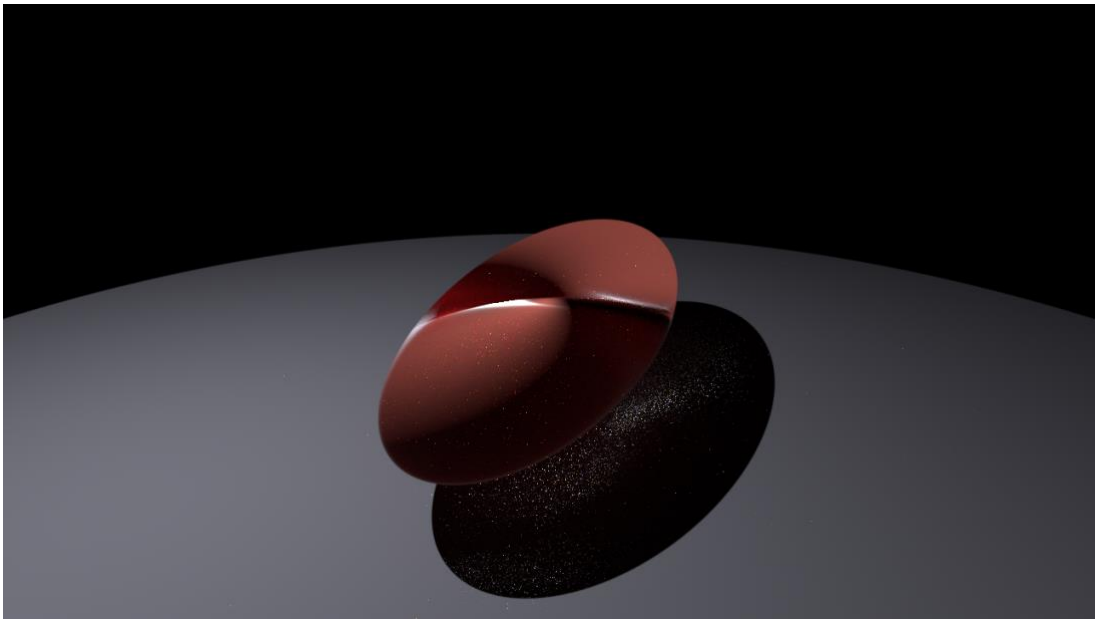
4. Rendered Result Without the Bug

Reference image (old/original binary): part_2/results/old.png



5. Rendered Result With the Bug

Bug image (modified binary): part_2/results/new.png

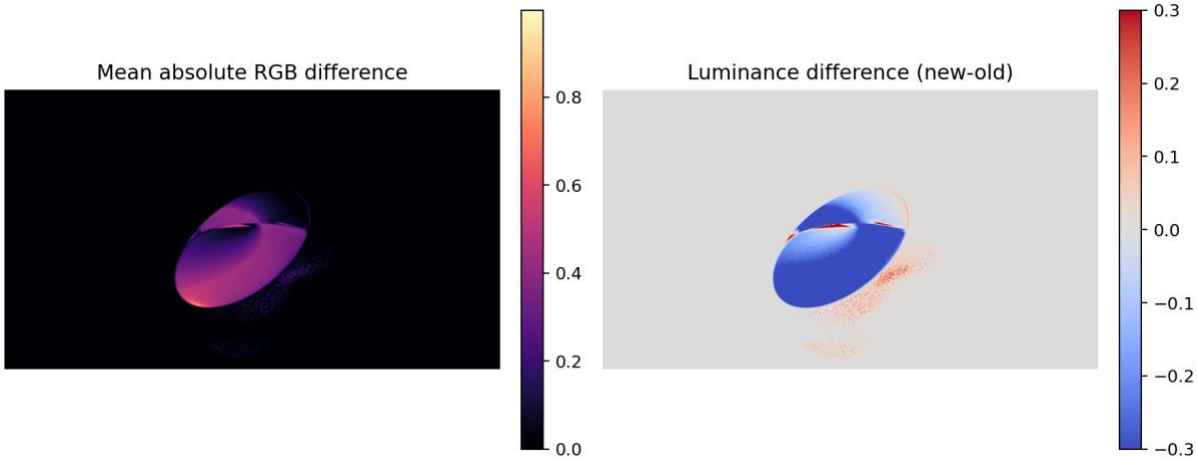


6. Result Analysis (Fact-based)

Image-level difference (old vs new): MAE=0.025548, RMSE=0.099222, max abs diff=1.000, pixels with >0.01 mean RGB difference=10.61%.

Global brightness changes from mean RGB 0.177199 (old) to 0.156012 (new), indicating systematic shading shift rather than random noise.

Difference visualization (computed from old.png and new.png):



Interpretation: the bug alters shading normals after non-uniform object scaling, so BRDF evaluation receives incorrect geometric orientation. Therefore the rendered image changes

in structured regions (surface highlights/shading transitions), matching the expected failure mode of normal-transform errors.

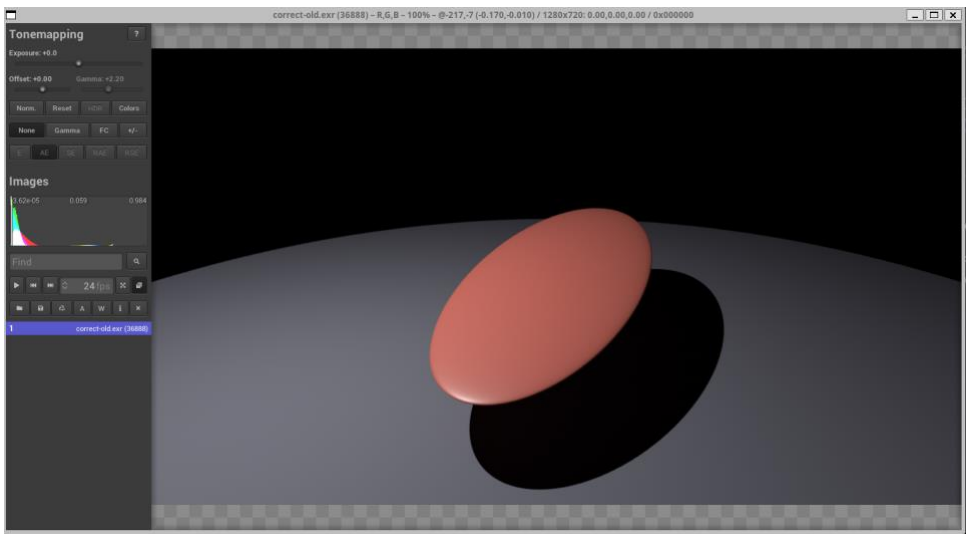
7. Conclusion

The experiment is controlled and reproducible: **only normal-transform code is changed**, scene/seed/sampler remain fixed. Results directly confirm that replacing inverse-transpose normal transform with direct matrix transform introduces physically incorrect shading under non-uniform scaling.

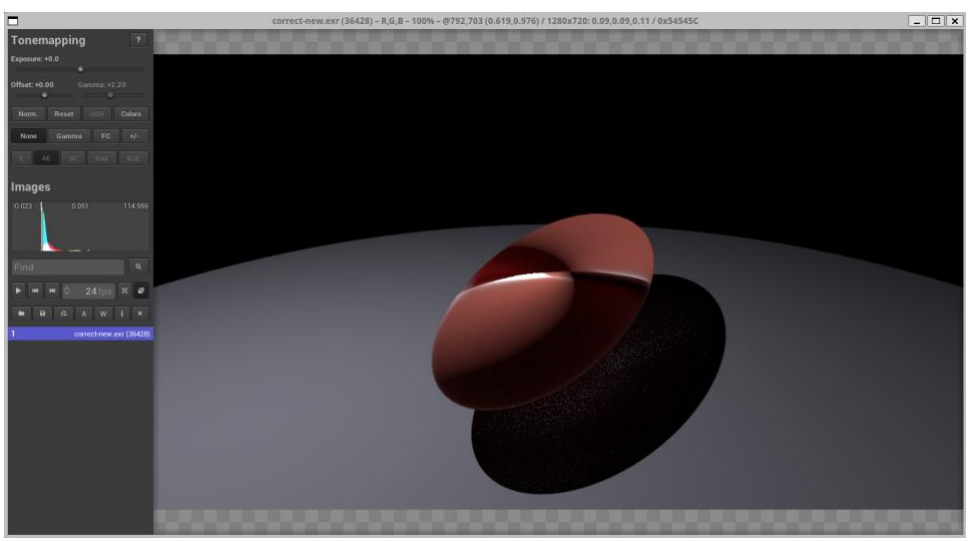
8. EXR Visualization in tev (Additional Evidence)

To avoid interpretation bias from 8-bit PNG output, EXR results were also inspected in tev. The following images are tev screenshots of old/new EXR frames.

Without bug (old EXR in tev):



With bug (new EXR in tev):



Concise comparison:

- The buggy version still shows the same structural shading error on the scaled sphere (incorrect highlight/banding and unrealistic orientation), so the bug conclusion is unchanged.
- Compared with direct PNG outputs, tev views show fewer obvious white/black speckles. This is expected because EXR keeps linear HDR radiance, while PNG is 8-bit output-referred data with quantization and display encoding.
- tev applies interactive tone mapping / exposure for display; this can suppress or redistribute extreme pixel contrast that appears more noticeable in PNG. (as also discussed in this GitHub issue)

Therefore, old/new consistency should be judged primarily by geometric shading patterns (shape of highlight and shadow transition), not only isolated bright/dark outlier pixels.